

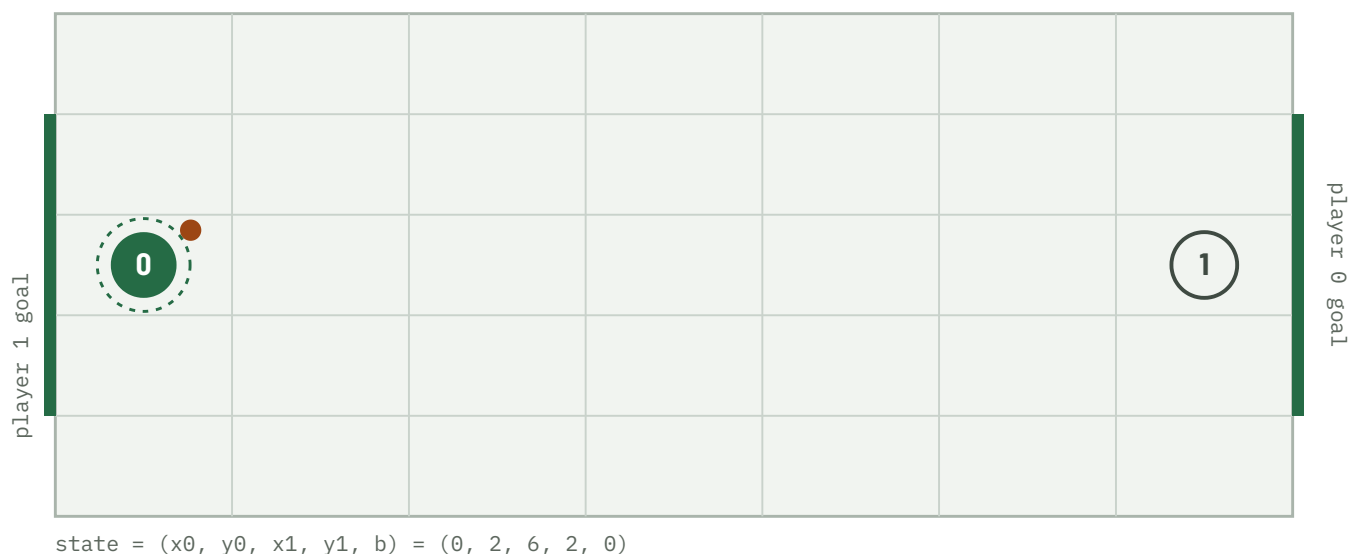
Pure Nash Equilibria for Soccer

Solving a two-player zero-sum Markov game with pure-strategy Nash Q-iteration — and finding where pure strategies quietly stop working.

RESEARCH QUESTION

Can pure-strategy Nash equilibria be used to *efficiently* solve the soccer Markov game, and under what state and reward settings does a pure Nash equilibrium exist?

repo · soccer-markov-nash | 131 tests | built on the CS 540 A10 assignment



The kickoff. 7×5 grid, 2 380 non-terminal states. Player 0 carries the ball and attacks the right edge; player 1 the left. Actions U D L R are chosen simultaneously; reward is +1 / -1 / 0, zero-sum.

THE ANSWER, IN FOUR LINES

- 1** Under the A10 move rule, a pure stationary Nash equilibrium exists at every state. Pure-strategy Nash Q-iteration solves the game exactly in 9 sweeps with no linear program at all.

- 2** A coin-flip tie-break does not break this. Randomising *who wins a contested square* leaves the value function bit-identical to the deterministic game's.

- 3** Littman's random *move order* does break it. On the 7×5 board, 94 of 2 380 stage games (3.9%) have no pure saddle, so no pure stationary equilibrium exists — and the effect is board-size dependent.

- 4** Potential-based reward shaping preserves the equilibrium exactly; a naive per-step possession bonus changes it. The recommended solver is the hybrid: pure where possible, LP only where necessary.

The two solvers

The game is zero-sum, so at every state the stage game is a payoff matrix $M(s) [a_0, a_1] = E[R + \gamma V(s')]$ and the Shapley (1953) fixed point is the minimax value:

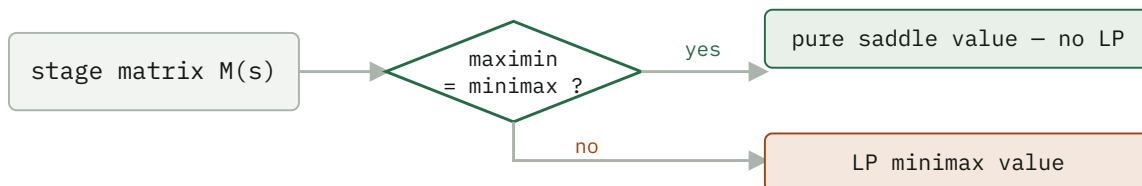
$$\begin{aligned} V(s) &= \text{val}(M(s)) = \max_n \min_{k \in \mathcal{K}} (p^T M) [a_1] \\ &= \min_p \max_{a_0} (Mq) [a_0] \end{aligned}$$

The $Q = R + \beta \text{Nash}(Q')$ update from the project notes is the same operator. Two stage solvers implement the two versions asked for:

SOCCER_NASH/MATRIX_GAMES.PY

VERSION	STAGE VALUE	METHOD
Pure Nash Q-iteration	pure maximin $\max_{a_0} \min_{k \in \mathcal{K}} M[i, j]$	no LP, $O(A^2)$
LP / Mixed Nash Q-iteration	LP minimax value	scipy linprog (HiGHS)
Hybrid ← default	pure saddle if maximin = minimax, else LP	exact everywhere

A pure saddle point exists *iff* the maximin equals the minimax. When it does, the pure value *is* the minimax value, so the hybrid is exact; when it doesn't, the hybrid falls back to the LP for that state only.



The hybrid solver. On the deterministic game the "yes" branch is taken at every one of the 2 380 states, so the LP is never called.

When does a pure Nash equilibrium exist?

By move rule – 7×5 board, $\gamma = 0.9$

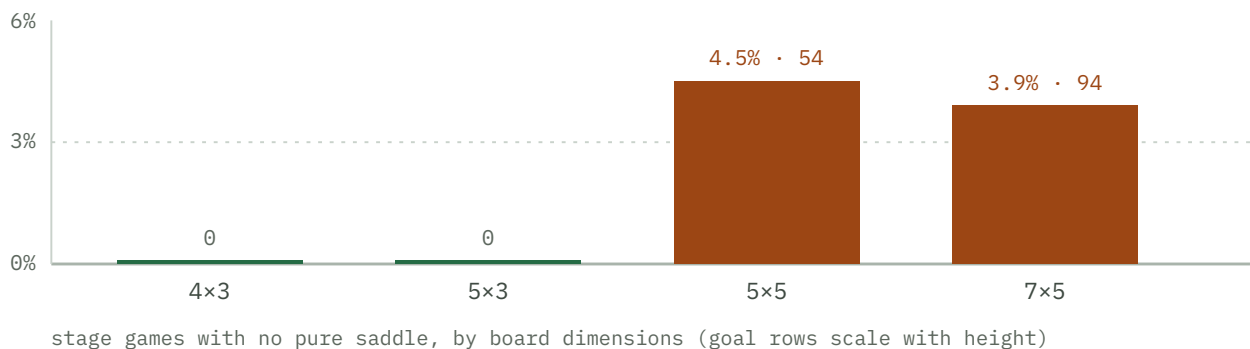
MOVE ORDER	STOCHASTIC	NO-SADDLE STAGE GAMES	PURE EQ?	V(KICKOFF)
deterministic	no	0 / 2380	yes, everywhere	0
coinflip tie-break	yes	0 / 2380	yes, everywhere	0
random move order	yes	94 / 2380	no	+0.150

KEY FINDING

The deterministic and coin-flip value functions are *identical* — $\max |V_{\text{det}} - V_{\text{coin}}| = 0$ for every γ . With optimal play neither player enters a contest it would lose deterministically, so the coin is never flipped on the equilibrium path.

It is **not stochasticity** that forces mixed strategies. It is the sequential coupling of Littman's random move order — where whether you steal the ball depends on who resolves first *and* on both players' targets — that creates matching-pennies sub-games with no pure saddle.

By board size – random move order



Room to fail. The no-saddle states are the "defender directly ahead of the carrier" stand-off; they come in exact mirror pairs and never occur when the players are far apart. Narrow boards have no room for that geometry, so their equilibrium stays pure.

By reward setting

Potential-based shaping $F(s, s') = \gamma\Phi(s') - \Phi(s)$ with Φ = possession bonus + ball advancement:

- $V_{\text{shaped}} = V_{\text{base}} - \Phi$ to machine precision; identical policies; identical no-saddle set. **The equilibrium — pure or mixed — is unchanged.**
- It *slows* exact value iteration ($9 \rightarrow \sim 130$ sweeps), because Φ is nonzero on the $\sim 1\,000$ states where $V_{\text{base}} = 0$.

CAUTION

A non-potential per-step possession bonus is **not** invariant: a $+0.05$ bonus turns the kickoff draw into a $+0.5$ value for the carrier and changes the optimal policy. Shaping rewards for this game must be potential-based.

Can pure equilibria solve it *efficiently*?

Yes. On the deterministic and coin-flip games the pure solver *is* the exact solver — pure, hybrid and mixed return the same value function in 9 sweeps with zero LP calls. On the random game the hybrid is still the right tool:

RANDOM MOVE ORDER, $\Gamma = 0.9$

SOLVER	SWEEPS	V(KICKOFF)	MATRIX-GAME SOLVES	WALL CLOCK
pure	18	+0.000	0	<1 s
hybrid exact	94	+0.150	~8 800	~20 s
mixed exact	94	+0.150	~3.6 M	~8 min

The hybrid reproduces the full-LP value function to $4e-16$ while solving an LP on only the ~4% of states that need one. The pure-only solver is a strict lower bound — it reports a kickoff draw when the game actually favours the initial carrier by 0.15, and sits up to 0.37 below the true value elsewhere. Efficient, but when a mixed equilibrium is the real answer, wrong.

Freeze then iterate – and its staleness

Solving a matrix game at every state on every sweep is wasteful. `run_policy_iteration()` solves each stage game only on its outer rounds and holds the strategies fixed for cheap linear evaluation sweeps in between — the notes' "don't recompute Nash every time".

RANDOM MOVE ORDER, HYBRID SOLVER – SAME FIXED POINT, $|DIFF| < 3E-9$

	ROUNDS / SWEEPS	MATRIX-GAME SOLVES	WALL CLOCK
value iteration	108	9 672	13.8 s
policy iteration	21	1 879	17.2 s

The solver records the **staleness** the scheme's proposer flagged — how far the frozen strategies are from the Nash of the current Q. It stays *maximal* (a full pure-strategy flip) for **16 outer rounds**, then snaps to $< 1e-8$. The thrashing eats the 5× saving in LP solves; on the deterministic game freeze-then-iterate is strictly worse. Value iteration with the per-sweep Nash cache wins here — the freeze trick pays off only where the equilibrium solve dominates the backup.

Numerical foundations

Three things the research meeting flagged as unsolved for the discrete solver (soccer_nash/numerics.py):

The value is three numbers

For strategies (p, q) the row player can *guarantee* $\min_k (pM)_k$, *gets* $p^T M q$, and can *reach at most* $\max_i (Mq)_i$. At equilibrium they coincide; numerically they never do. `value_bracket` returns all three; the true minimax value is bracketed, so the width certifies the LP error.

FINDING

With `scipy`'s HiGHS solver the three numbers agree to $1.1e-16$ per stage game and $8.7e-10$ accumulated. The concern is real for older vertex/simplex solvers but not a practical problem here — the LP solution is a certified $1e-16$ -equilibrium.

Rounding to force indifference is scale-blind

Mixed equilibria need *exact* indifference, so RPS-plus-epsilon has no exact pure saddle — but rounding manufactures one, and under discounting the real γ^k differences are that small. `classify_stage_game` uses a tolerance that scales with the entries.

MOVE ORDER	PURE	DEGENERATE	MIXED	ROUNDING TO 0.1 FLIPS A SADDLE
deterministic	0	2380	0	0
random	604	1682	94	62

Rounding to 0.1 misclassifies 62 of the random game's stage games — the small-entry region where mixing lives. The deterministic game is safe to round but has a **non-strict saddle at every one of its 2 380 states**: the pure equilibrium exists everywhere yet is tie-ridden.

The mixed states are matching pennies

Of the 94 states that require mixing, **68 reduce to a 2×2 support**: carrier {advance, hold} × defender {block, intercept} — a "guess where the ball goes" game. That is why Littman's *random move order* creates the mixing and the deterministic A10 rule does not.

The equilibrium policy is worth playing

- The Nash Q policy has a duality gap $V0_br(s_\theta) + V1_br(s_\theta) < 1e-9$ for every move order — **no opponent can beat the game value**.
- Nash vs. Nash reproduces the value: forced draws in the deterministic and coin-flip games; a $+0.157$ empirical discounted return (vs. $+0.150$ computed) in the random game.
- Best-responding to a *guess* about the opponent is fragile: the Part 2 best-response-to-the-scripted-opponent policy has exploitability 0.43 — worse than moving uniformly at random (0.39) once the opponent best-responds to it.

That last line is exactly the A10 competition's warning about non-equilibrium submissions. The equilibrium networks (`scripts/a10_competition.py`) are the principled answer — though a neural approximation is only as safe as its worst-fit state, so the script prints the trained network's exploitability and the student re-seeds until it is near zero.

Conclusion

For the soccer game as specified by A10, pure-strategy Nash equilibria are not just usable but **sufficient and efficient**: they exist everywhere and are found in a handful of sweeps with no linear programming. Pure equilibria fail only under Littman's sequential random move order, and even then only on ~4% of states of a large enough board. The hybrid solver — pure where possible, LP where necessary — is exact everywhere and pays the LP cost only where a pure equilibrium genuinely does not exist. Potential-based reward shaping leaves all of this untouched.

```
soccer_nash — game.py · matrix_games.py · nash_q.py · numerics.py · best_response.py ·
mlp.py · opponents.py · shaping.py · exploit.py · simulate.py
scripts — analyze.py · numerics.py · policy_iteration.py · selfplay.py · a10_part1.py ·
a10_part2.py · a10_competition.py
131 passing tests (126 fast + 5 slow) | env: 7×5 grid, 2380 states, zero-sum ±1, γ = 0.9
```